Apellidos: Gómez González
Nombre: Isaac
Asignatura: Programación de Concurrencia y Distribuida
Grupo: 2 **Subgrupo:** 3
Convocatoria:
Año académico: 2025–2026
Profesor: Adrián Canovas Rodríguez

MEMORIA DE PRÁCTICAS

Entregables 2025–2026

Índice

1. Ejercicio 1: ReentrantLock **2**
1.1. Recursos no compartibles . . . 3
1.2. Condiciones de sincronización . . . 3
1.3. Pseudocódigo comentado . . . 3
1.4. Cuestiones planteadas en los ejercicios . . . 5
1.5. Código Java de los ejercicios comentado . . . 6

2. Ejercicio 2: Semáforos **9**
2.1. Recursos no compartibles . . . 10
2.2. Condiciones de sincronización . . . 10
2.3. Pseudocódigo comentado . . . 10
2.4. Cuestiones planteadas en los ejercicios . . . 10
2.5. Código Java de los ejercicios comentado . . . 11

3. Ejercicio 3: Monitores **12**
3.1. Recursos no compartibles . . . 13
3.2. Condiciones de sincronización . . . 13
3.3. Pseudocódigo comentado . . . 13
3.4. Cuestiones planteadas en los ejercicios . . . 13
3.5. Código Java de los ejercicios comentado . . . 13

4. Ejercicio 4: Paso de Mensajes Asíncrono **14**
4.1. Recursos no compartibles . . . 15
4.2. Condiciones de sincronización . . . 15
4.3. Pseudocódigo comentado . . . 15
4.4. Cuestiones planteadas en los ejercicios . . . 15
4.5. Código Java de los ejercicios comentado . . . 15

1. Ejercicio 1: ReentrantLock

Supongamos dos hilos con la siguiente funcionalidad en un **centro de alto rendimiento**:

- Un primer hilo genera una matriz **A** (3x3) con enteros que representa métricas de entrenamiento (por ejemplo: fuerza, velocidad y resistencia en 3 días), calcula el "impacto acumulado" $\mathbf{A^2 = A \cdot A}$ e imprime:

  A x A
  a1 a2 a3
  a4 a5 a6
  a7 a8 a9

  x

  a1 a2 a3
  a4 a5 a6
  a7 a8 a9

  A2

  r1 r2 r3
  r4 r5 r6
  r7 r8 r9

- El otro hilo hace lo mismo, pero calcula **A + A** (doble carga) e imprime:

  A + A

  ...
  2A
  ...

Cada hilo repite su actividad **10 veces**. Desarrollar un programa concurrente en Java. La sincronización necesaria se resolverá usando cerrojos ReentrantLock.

1.1. Recursos no compartibles

Unicamente la pantalla, ya que es el recurso que ambos hilos necesitan para imprimir sus resultados.

1.2. Condiciones de sincronización

El uso de un cerrojo para bloquear la sección crítica de la pantalla. Que permite a los hilos el acceso a la pantalla en exclusión mutua, evitando así que ambos hilos impriman al mismo tiempo y mezclen sus salidas.

1.3. Pseudocódigo comentado

```
Algoritmo: Clase Main
// Variables globales
Datos:
cerrojoPantalla : Cerrojo
NUM_OPERACIONES : Entero
LONGITUD_FILAS_Y_COLUMNAS : Entero
// Inicialización de variables globales
NUM_OPERACIONES ← 10
LONGITUD_FILAS_Y_COLUMNAS ← 3
Función main()
    initial (cerrojoPantalla, 1)
    cobegin
        suma()
        multiplicar()
    coend
fin
```

```
Algoritmo: Clase SumaMatriz
// Variables de instancia
Datos:
metricasDeEntrenamiento : Entero[][]
impactoAcumulado : Entero[][]

Función suma()
    para i ← 1 a NUM_OPERACIONES hacer
        generarMatriz (metricasDeEntrenamiento)
        // Calcular suma de matrices
        // Guardar resultado en impactoAcumulado

        wait(cerrojoPantalla)
        // Impresión
        signal(cerrojoPantalla)
    fin
fin
```

```
Algoritmo: Clase CuadradoMatriz
// Variables de instancia
Datos:
metricasDeEntrenamiento : Entero[][]
impactoAcumulado : Entero[][]

Función multiplicar()
    para i ← 1 a NUM_OPERACIONES hacer
        generarMatriz (metricasDeEntrenamiento)
        // Calcular cuadrado de matrices
        // Guardar resultado en impactoAcumulado

        wait(cerrojoPantalla)
        // Impresión
        signal(cerrojoPantalla)
    fin
fin
```

1.4. Cuestiones planteadas en los ejercicios

a) **Si no usas sincronización, ¿cómo podría ser la salida por pantalla?**

Como regla general, si no usas sincronización, la salida por pantalla es caótica e impredecible. En este caso, dado que ambos hilos (SumaMatriz y CuadradoMatriz) están accediendo a la pantalla para imprimir sus resultados, sin sincronización podrían intercalarse las impresiones de ambos hilos. Esto podría resultar en una salida mezclada donde partes de la matriz de SumaMatriz se imprimen junto con partes de la matriz de CuadradoMatriz, haciendo que sea difícil o imposible entender los resultados.

En el caso particular de java cada System.out.println() es una operación atómica, lo que significa que cada llamada a esta función se ejecutará completamente sin ser interrumpida por otro hilo. Sin embargo, esto no garantiza que las impresiones de ambos hilos no se mezclen, ya que cada hilo podría imprimir varias líneas de la matriz antes de que el otro hilo tenga la oportunidad de imprimir. Por lo tanto, aunque cada línea individual se imprima de manera atómica, la secuencia completa de impresiones podría resultar en una salida caótica si no se sincroniza adecuadamente.

1.5. Código Java de los ejercicios comentado

Clase Main

```java
package ejercicio1;

import java.util.Random;
import java.util.concurrent.locks.ReentrantLock;

public class Main {
    public static final int NUM_OPERACIONES = 10;
    public static final int LONGITUD_FILAS_Y_COLUMNAS = 3;
    public static final int NUM_0_9 = 10;

    /* Este cerrojo se encarga de bloquear la sección crítica de la pantalla para que no acceda
    * más de un hilo a la vez */
    public static ReentrantLock cerrojoPantalla = new ReentrantLock();

    /**
    * Función que le da valores aleatorios entre 0 y 9 a una matriz de 3x3,
    * @param matriz Es la matriz que se rellenara con núevos valores aleatorios
    */
    public static void generarMatriz(int[][] matriz){
        for(int i = 0; i < LONGITUD_FILAS_Y_COLUMNAS; i++) {
            for(int j = 0; j < LONGITUD_FILAS_Y_COLUMNAS; j++) {
                Random random = new Random();
                matriz[i][j] = random.nextInt(NUM_0_9);
            }
        }
    }


    /**
    * Función principal del programa donde se ejecutan dos hilos, uno que se encarga de
    * hacer el cuadrado de una matriz y otro el doble de una matriz.
    * @param args
    */
    public static void main(String[] args) {
        CuadradoMatriz cuadrado = new CuadradoMatriz();
        SumaMatriz suma = new SumaMatriz();
        cuadrado.start();
        suma.start();

        try {
            cuadrado.join();
            suma.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Clase SumaMatriz

```
package ejercicio1;

public class SumaMatriz extends Thread{


    @Override
    public void run() {

        int metricasDeEntrenamiento[][] =
            new int[Main.LONGITUD_FILAS_Y_COLUMNAS][Main.LONGITUD_FILAS_Y_COLUMNAS];
        int impactoAcumulado[][] =
            new int[Main.LONGITUD_FILAS_Y_COLUMNAS][Main.LONGITUD_FILAS_Y_COLUMNAS];

        for (int operacionesRealizadas = 0;
            operacionesRealizadas < Main.NUM_OPERACIONES; operacionesRealizadas++) {

            Main.generarMatriz(metricasDeEntrenamiento);

            // Suma de la matriz A + A (2A)
            for (int i = 0; i < Main.LONGITUD_FILAS_Y_COLUMNAS; i++) {
                for (int j = 0; j < Main.LONGITUD_FILAS_Y_COLUMNAS; j++) {
                    impactoAcumulado[i][j] =
                        metricasDeEntrenamiento[i][j] + metricasDeEntrenamiento[i][j];
                }
            }


            /* Aquí se bloquea el cerrojo de la pantalla para asegurar que este recurso no
            * compartible no sea usado por más de un proceso a la vez
            */
            Main.cerrojoPantalla.lock();

            System.out.println("A␣+␣A\n");

            for (int[] fila : metricasDeEntrenamiento) {
                for (int valor : fila) {
                    System.out.print(valor + "a␣");
                }
                System.out.println();
            }
            System.out.println("\n+\n");

            for (int[] fila : metricasDeEntrenamiento) {
                for (int valor : fila) {
                    System.out.print(valor + "a␣");
                }
                System.out.println();
            }

            System.out.println("\n2A\n");

            for (int[] fila : impactoAcumulado) {
                for (int valor : fila) {
                    System.out.print(valor + "r␣");
                }
                System.out.println();
            }
            System.out.println();

            Main.cerrojoPantalla.unlock();

        }

    }
}
```

Clase CuadradoMatriz

```
package ejercicio1;

public class CuadradoMatriz extends Thread {

    @Override
    public void run() {

        int metricasDeEntrenamiento[][] =
            new int[Main.LONGITUD_FILAS_Y_COLUMNAS][Main.LONGITUD_FILAS_Y_COLUMNAS];
        int impactoAcumulado[][] =
            new int[Main.LONGITUD_FILAS_Y_COLUMNAS][Main.LONGITUD_FILAS_Y_COLUMNAS];

        for (int operacionesRealizadas = 0;
            operacionesRealizadas < Main.NUM_OPERACIONES; operacionesRealizadas++) {

            Main.generarMatriz(metricasDeEntrenamiento);

            // Cálculo del cuadrado de la matriz
            for (int i = 0; i < Main.LONGITUD_FILAS_Y_COLUMNAS; i++) {
                for (int j = 0; j < Main.LONGITUD_FILAS_Y_COLUMNAS; j++) {
                    impactoAcumulado[i][j] = 0;
                    for (int k = 0; k < Main.LONGITUD_FILAS_Y_COLUMNAS; k++) {
                        impactoAcumulado[i][j] +=
                            metricasDeEntrenamiento[i][k] * metricasDeEntrenamiento[k][j];
                    }
                }
            }


            /*
            * Aquí se bloquea el cerrojo de la pantalla para asegurar que este recurso no
            * compartible no sea usado por más de un proceso a la vez
            */
            Main.cerrojoPantalla.lock();

            System.out.println("A␣x␣A\n");

            for (int[] fila : metricasDeEntrenamiento) {
                for (int valor : fila) {
                    System.out.print(valor + "a␣");
                }
                System.out.println();
            }
            System.out.println("\nx\n");

            for (int[] fila : metricasDeEntrenamiento) {
                for (int valor : fila) {
                    System.out.print(valor + "a␣");
                }
                System.out.println();
            }

            System.out.println("\nA2\n");

            for (int[] fila : impactoAcumulado) {
                for (int valor : fila) {
                    System.out.print(valor + "r␣");
                }
                System.out.println();
            }
            System.out.println();

            Main.cerrojoPantalla.unlock();

        }

    }
}
```

2. Ejercicio 2: Semáforos

En un **centro de evaluación deportiva**, hay **3 pantallas** (paneles) para mostrar resultados de pruebas físicas. En el sistema se dispone de **3 instancias** de la clase Panel proporcionada en los recursos del aula virtual. Hay **10 hilos**, cada uno representa a un **equipo** de **4 deportistas**. Cada hilo repite **3 veces** la siguiente secuencia:

1) Cada hilo genera aleatoriamente una **matriz 5x5** de enteros para cada **uno de los 4 deportistas del equipo**, donde **las filas representan 5 ejercicios** del test físico (sprint, salto, lanzamiento, resistencia, y agilidad) y **las columnas representan 5 intentos** para conseguir la mejor marca en cada ejercicio.

2) Para cada deportista, se calcula su **mejor marca por ejercicio**, seleccionando el **máximo** (o mínimo, en caso de ser tiempo) de los 5 intentos de cada fila. El resultado para cada deportista es un **vector de 5 elementos** (uno por ejercicio).

3) A partir de los resultados anteriores, se construye una **matriz resumen de 4x5 llamada R**, que representa el resumen del equipo:

   - Columna 1: mejor marca de cada deportista para el ejercicio 1
   - Columna 2: mejor marca de cada deportista para el ejercicio 2
   - Columna 3: mejor marca de cada deportista para el ejercicio 3
   - Columna 4: mejor marca de cada deportista para el ejercicio 4
   - Columna 5: mejor marca de cada deportista para el ejercicio 5

4) Muestra en **un panel que esté disponible** la siguiente salida:

   Usando panel P el hilo (equipo) id

   Matriz R (resumen del equipo X: mejores marcas) = ...

   Terminando de usar panel P el hilo (equipo) id

   Los pasos 1, 2, 3 y 4 se repiten **3 veces** por equipo.

   Los 10 hilos (equipos) comparten los **3 paneles** del sistema y, usando **semáforos**, se debe añadir la sincronización necesaria para que:

   - nunca dos hilos escriban simultáneamente en el mismo panel,

- el uso de los paneles sea eficiente (máximo paralelismo posible).

La sincronización hay que resolverla **con semáforos**.

2.1. Recursos no compartibles

2.2. Condiciones de sincronización

2.3. Pseudocódigo comentado

Algoritmo: Clase Main

Datos:
paneles : *Panel*[]
NUMERO_DE_PANELES : *Entero*
NUMERO_DE_HILOS : *Entero*
NUMERO_DE_DEPORTISTAS_POR_HILO : *Entero*
NUMERO_DE_SECUENCIAS : *Entero*

// Inicialización de variables globales
NUMERO_DE_PANELES ← 3
NUMERO_DE_HILOS ← 10
NUMERO_DE_DEPORTISTAS_POR_HILO ← 4
NUMERO_DE_SECUENCIAS ← 3
NUMERO_DE_EJERCICIOS ← 5

Algoritmo: Clase

Datos:
matrizDeEjercicios : *Entero*[][]
R : *Entero*[][]

Algoritmo: Clase

2.4. Cuestiones planteadas en los ejercicios

Según las condiciones del problema:

a) **(0.5 puntos sobre 10) Si consideramos un panel solamente ¿cómo es el acceso al mismo?** Si consideramos los tres paneles ¿cómo es el acceso? Explica ambas situaciones.

b) **(0.5 puntos sobre 10) ¿Qué tipo de semáforos has usado para resolver el problema (binarios, generales)?**

2.5. Código Java de los ejercicios comentado

Clase

3. Ejercicio 3: Monitores

En este ejercicio se quiere simular mediante un programa concurrente la actividad en un **gimnasio**. El flujo para entrenar es el siguiente:

1. **Acceso al gimnasio mediante tornos**): Un cliente llega al gimnasio y debe pasar por **uno de los 3 tornos de acceso** para validar su entrada. Cada torno tiene su **cola individual**. El tiempo que tarda el cliente en completar el acceso (validación + entrada) es **X milisegundos** (valor aleatorio entre 1 y 5). Nota: el cliente debe situarse en el **primer torno** que encuentre libre; si los 3 están ocupados, esperará en la **cola del torno** que tenga menor tiempo de espera.

2. **Elección de zona de entrenamiento**: Tras pasar la zona de tornos, el cliente elige a qué **zona de entrenamiento** dirigirse en función del tiempo estimado de espera. El gimnasio tiene **4 zonas**, y **cada zona dispone de 5 máquinas: Zona 1 – Cardio, Zona 2 – Fuerza, Zona 3 – Funcional, Zona 4 - Estiramientos** . Un cliente **puede entrar** a una zona si al menos una de sus máquinas está libre. Si hay varias zonas con máquinas libres elegirá una de forma aleatoria. Si **las 5 máquinas** de una zona están ocupadas, el cliente debe esperar en la **cola de acceso de esa zona** (una cola por zona).

3. Dentro de la **zona de Cardio** existe una **bicicleta premium** que tiene una **cola exclusiva propia**, independiente de la cola general de la zona. Un cliente puede entrar en Cardio si hay alguna de las 5 máquinas libres (según el paso 2). Si el cliente desea usar la **bicicleta premium** (decisión con probabilidad del 30esperar en la **cola exclusiva** de la bicicleta premium, incluso aunque ya esté dentro.

4. El entrenamiento tarda **Y milisegundos** (valor aleatorio por cliente), representado mediante el método Sleep() de Java.

Se realizará una simulación con **50 hilos cliente**, con valores X e Y inicializados aleatoriamente. Desarrollar un programa concurrente en Java para resolver el problema anterior usando monitores como mecanismo para la sincronización.

Formato de salida obligatorio: Justo antes de decidir a qué zona dirigirse, cada cliente debe imprimir en pantalla:

Cliente id ha pasado por el torno: —-
Tiempo en el torno (acceso): X
Zona elegida: —-
Tiempo de entrenamiento: Y
Estimación de espera (sin incluirse a sí mismo):
Zona1(Cardio)=—-, Zona2(Fuerza)=—-, Zona3(Funcional)=—-, Zona4(Estiramientos)=—-
Espera bicicleta premium (si aplica)=—-

3.1. Recursos no compartibles

3.2. Condiciones de sincronización

3.3. Pseudocódigo comentado

Algoritmo: `Clase`

3.4. Cuestiones planteadas en los ejercicios

Según las condiciones de sincronización del problema,

a) **(0.5 puntos) Indica si la acción de entrenar es concurrente o en exclusión mutua y justifica.**

b) **(0.75 puntos) ¿Qué tipo de monitor Java has usado? Justifica.**

c) **(0.5 puntos) En el monitor, ¿has usado notify/signal o notifyAll/signalAll? Justifica**

d) **(0.75 puntos) ¿Cómo se ha resuelto la exclusión mutua de la pantalla en este problema**

3.5. Código Java de los ejercicios comentado

Clase

4. Ejercicio 4: Paso de Mensajes Asíncrono

Consideremos un conjunto de 50 aficionados que llegan a un estadio de fútbol para entrar a un partido y deben pasar por el control de acceso tras validar su entrada. Para acceder hay dos tornos con sus respectivas colas: la cola R y la cola L, siendo la cola R más rápida que la L (por ejemplo, porque el torno R tiene lector más eficiente o un agente adicional). Cada aficionado tiene que usar una cola para validar su entrada durante un tiempo estimado (valor aleatorio entre 1 y 10). El controlador de accesos, cuando llega un nuevo aficionado a validar la entrada, estima su tiempo de validación y a aquellos aficionados cuyo tiempo estimado sea menor o igual a 5, se les asignará el torno más rápido, es decir, la cola R, a los demás aficionados se les asignará el torno L. Durante el partido, cada aficionado puede salir de su grada para ir a los baños o a la zona de compra de comida y bebida. Para volver a su zona, debe pasar de nuevo por los tornos, repitiendo el proceso de validación. Este ciclo de "salir y volver a entrar" lo realizará 5 veces a lo largo del partido. Es decir, cada hilo repite 5 veces la siguiente secuencia de acciones:

1. Realiza una acción previa (usando Sleep ()), por ejemplo caminar hacia los baños.
2. Solicita ponerse en una cola (R o L según el controlador de accesos le indique).
3. Realiza la validación en el torno asignado.
4. Libera la cola (deja libre el torno),
5. Imprime en pantalla la información que se indica a continuación:

   "Aficionado id ha usado la cola X

   Tiempo de validación = T

   Thread.sleep(T)

   Aficionado id liberando la cola X"

   donde X es la cola asignada y T el tiempo de validación.

Desarrollar un programa concurrente en Java para resolver el problema anterior usando como mecanismo para la sincronización el paso de mensajes asíncrono.

4.1. Recursos no compartibles

4.2. Condiciones de sincronización

4.3. Pseudocódigo comentado

Algoritmo: `Clase`

4.4. Cuestiones planteadas en los ejercicios

a) **(0.5 puntos sobre 10) ¿Se pueden usar simultáneamente los dos tornos? Justifica la respuesta**

b) **(0.5 puntos sobre 10) ¿Cómo has resuelto la exclusión mutua de la pantalla?**

4.5. Código Java de los ejercicios comentado

Clase